Advanced ASP.NET Web API Features

1. Advanced API Features in ASP.NET Core

1.1 Attribute Routing

- Enables defining routes directly on controllers/actions.

```
[Route("api/[controller]")]
[ApiController]
public class ProductsController : ControllerBase
{
   [HttpGet("{id}")]
   public IActionResult Get(int id) { }
}
```

- Supports:
  - Route parameters ({id})
  - Constraints ({id:int})
  - Optional params ({id?})

1.2 Model Binding & Validation

Model Binding

- Maps HTTP request data → C# objects
- Sources:
  - Route
  - Query string
  - Body
  - Headers

```
public IActionResult Create(Product product){  ...  }
```

Validation

- Uses Data Annotations:

```
public class Product
{
   [Required]
   public string Name { get; set; }

   [Range(1, 1000)]
   public decimal Price { get; set; }
}
```

- Automatic validation with [ApiController]

1.3 Filters (Cross-Cutting Concerns)

Types of filters:

- Authorization Filters
- Action Filters
- Exception Filters
- Result Filters

Example:

```
public class MyActionFilter : IActionFilter
{
    public void OnActionExecuting(ActionExecutingContext context) { }
    public void OnActionExecuted(ActionExecutedContext context) { }
}
```

Use cases:

- Logging
- Validation
- Error handling

1.4 Dependency Injection (DI)

- Built-in IoC container

```
// Register in Program.cs
builder.Services.AddScoped<IProductService, ProductService>();

//Inject into controllers:
public ProductsController(IProductService service)
{
    _service = service;
}
```

1.5 Middleware

- Handles request/response pipeline

```
app.Use(async (context, next) =>
{
    Console.WriteLine("Request incoming");
    await next();
    Console.WriteLine("Response outgoing");
});
```

Common middleware:

- Authentication
- Authorization
- Logging
- Exception handling

1.6 API Versioning

- Supports multiple API versions

```
//Install package:
Microsoft.AspNetCore.Mvc.Versioning

// Example:
[ApiVersion("1.0")]
[Route("api/v{version:apiVersion}/products")]
```

1.7 Content Negotiation

- Returns data in requested format (JSON, XML)

```
builder.Services.AddControllers().AddXmlSerializerFormatters();
```

- Uses Accept header

1.8 Pagination, Sorting, Filtering

Example:

```
[HttpGet]
public IActionResult Get(int page = 1, int pageSize = 10)
```

Best practices:

- Limit max page size
- Return metadata (total records, pages)

1.9 Authentication & Authorization

Authentication

- Verifies identity (JWT, cookies)

Authorization

- Checks permissions

```
[Authorize]
public IActionResult GetSecureData()
```

1.10 CORS (Cross-Origin Resource Sharing)

Definition: CORS (Cross-Origin Resource Sharing) is a browser security mechanism that allows a server (Web API) to indicate which other origins—defined by domain, protocol, or port—are permitted to load its resources.

```
// 1.  Install the package:
Microsoft.AspNet.WebApi.Cors

// 2. Add CORS as service
builder.Services.AddCors(options =>
{
   options.AddPolicy("AllowAll",
      policy => policy.AllowAnyOrigin()
                  .AllowAnyMethod()
                  .AllowAnyHeader());
});

// 3. Use cors middleware
app.UseCors("MyAllowSpecificOrigins");
```

Note: It is very important to allow front-end frameworks like React, Angular (client applications) to interact with API

API Documentation and Testing

2.1 Swagger / OpenAPI

- Automatically generates API documentation

Install:

Swashbuckle.AspNetCore

Setup:

```
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();
```

Enable:

```
app.UseSwagger();
app.UseSwaggerUI();
```

Features:

- Interactive UI
- Test endpoints directly
- Shows request/response models

2.2 Annotations for Better Docs

```
/// <summary>
/// Gets a product by ID
/// </summary>
[HttpGet("{id}")]
public IActionResult Get(int id)
```

Enhancements:

- XML comments
- Data annotations
- Response types

2.3 API Testing Tools

Postman

- GUI-based API testing
- Supports:
  - GET, POST, PUT, DELETE
  - Headers, auth, body
  - Collections & automation

2.4 Integration Testing

- Tests full request pipeline

```
public class ProductsTests : IClassFixture<WebApplicationFactory<Program>>
{
   private readonly HttpClient _client;

   public ProductsTests(WebApplicationFactory<Program> factory)
   {
      _client = factory.CreateClient();
   }
   [Fact]
   public async Task Get_ReturnsSuccess()
```

```
  {
    var response = await _client.GetAsync("/api/products");
    response.EnsureSuccessStatusCode();
  }
}
```

2.5 Unit Testing APIs

- Tests individual components

Tools:

- xUnit
- Moq

Example:

```
var mockService = new Mock<IProductService>();
mockService.Setup(s => s.GetAll()).Returns(new List<Product>());
```

2.6 Logging & Monitoring

- Built-in logging framework
  _logger.LogInformation("Fetching products");

- Can integrate with:
  - Application Insights
  - Serilog

2.7 Error Handling & Status Codes

Best practices:

- Use correct HTTP status codes:
  - 200 OK
  - 201 Created
  - 400 Bad Request
  - 404 Not Found
  - 500 Internal Server Error

Example:

```
return NotFound();
```

Summary

Advanced API Features:

- Routing, Filters, Middleware
- DI, Authentication, Versioning
- Pagination, CORS, Content Negotiation

Documentation & Testing:

- Swagger/OpenAPI for documentation
- Postman & curl for testing
- Unit + Integration testing
- Logging and error handling